

Object Detection Performance Analysis

1. Introduction

This document outlines the approach and implementation details for performing object detection using the Ultralytics YOLO (You Only Look Once) model on a set of images. The primary objective of this analysis is to evaluate the model's performance in terms of speed and accuracy for detecting various objects within images.

2. Approach

2.1 Data Preparation

- **Image Input:** The code scans a specified directory for image files (in this case, JPEG images) that will be used for object detection.
- **Class Names:** A file named coco.names containing the names of the classes that the YOLO model can detect is loaded. Each line corresponds to a different object class.

2.2 Object Detection

- **Detection Function:** The detect_objects function is defined to process each image:
 - **Image Loading:** Each image is loaded using OpenCV.
 - **Color Conversion:** The image is converted from BGR to RGB format to match the input requirement of the YOLO model.
 - **Model Prediction:** The Ultralytics YOLO model predicts the objects in the image. I opted to use the Ultralytics implementation instead of TensorFlow due to issues encountered while using TensorFlow, which hindered the detection process. The Ultralytics model provided a more straightforward and efficient solution.
 - **Results Processing:** For each detected object, bounding boxes and labels are drawn on the image.

2.3 Performance Metrics

- **Inference Timing:** The time taken for preprocessing, inference, and postprocessing of each image is measured and displayed.
- **Output:** The processed images are saved with bounding boxes and labels, and displayed using Matplotlib.

3. Code Implementation

The following is the complete implementation of the object detection process:

```

python
Copy code
import cv2
import matplotlib.pyplot as plt
import glob
import os

# Load class names
with open('/content/coco.names', 'r') as f:
    classes = [line.strip() for line in f.readlines()]

# Get image paths
image_paths = glob.glob('/content/*.jpg')
print("Found images:", image_paths)

# Function to detect objects in an image
def detect_objects(image_path):
    image = cv2.imread(image_path)
    if image is None:
        print(f'Error loading image: {image_path}')
        return None

    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    results = yolo_model.predict(source=image_rgb)

    print(f"Detected {len(results)} results in {image_path}")

    for result in results:
        for box in result.bboxes:
            x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().astype(int)
            conf = box.conf[0].cpu().numpy()
            cls = box.cls[0].cpu().numpy()

            if 0 <= int(cls) < len(classes):
                label = f"{classes[int(cls)]}: {conf:.2f}"
            else:
                print(f"Warning: Class ID {int(cls)} not found in coco.names")
                label = f"Class {int(cls)}: {conf:.2f}"

            # Draw bounding box and label
            cv2.rectangle(image_rgb, (x1, y1), (x2, y2), (0, 255, 0), 3)
            cv2.putText(image_rgb, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255, 0, 0), 3)

    return image_rgb

```

```

# Create output directory for detected images
output_dir = '/content/output_images1'
os.makedirs(output_dir, exist_ok=True)

# Process images
for img_path in image_paths[:3]:
    detected_image = detect_objects(img_path)
    if detected_image is None:
        continue

    # Save the detected image to output directory
    output_image_path = os.path.join(output_dir, os.path.basename(img_path))
    cv2.imwrite(output_image_path, cv2.cvtColor(detected_image,
cv2.COLOR_RGB2BGR))

    # Display detected image
    plt.figure(figsize=(10, 10))
    plt.imshow(detected_image)
    plt.axis('off')
    plt.title(f'Detected Objects in {img_path}')
    plt.show()

```

4. Observations

The following observations were made during the execution of the object detection process:

1. **Image Detection:** The model successfully detected objects in all processed images. The detected objects included items like chairs, couches, and plants, with bounding boxes accurately drawn around them.
2. **Inference Times:**
 - The time taken for preprocessing, inference, and post processing varied per image. Here are the average timings observed:
 - **Image 1:**
 - Preprocess: 4.4 ms
 - Inference: 155.8 ms
 - Postprocess: 1.5 ms
 - **Image 2:**
 - Preprocess: 4.4 ms
 - Inference: 225.8 ms
 - Postprocess: 1.2 ms
 - **Image 3:**
 - Preprocess: 3.4 ms
 - Inference: 214.1 ms
 - Postprocess: 1.2 ms

- These timings indicate that the Ultralytics YOLO model is capable of performing real-time detections, especially with lower-resolution images.
- 3. **Scalability:** Given the speed at which detections are made, the approach can be scaled up to larger datasets or real-time video streams with minimal modifications.
- 4. **Accuracy:** The confidence levels for the detected objects were shown in the labels, providing insights into the reliability of the detections.

5. Conclusion

The object detection implementation using the Ultralytics YOLO model demonstrates effective performance with timely detections across various images. The results showcase the model's ability to accurately identify multiple objects, making it suitable for applications in real-time scenarios such as surveillance, self-driving cars, and automated quality control systems.